

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



TRẦN MẠNH DUY

**XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO
VÀ ỨNG DỤNG CHỮ KÝ SỐ
KHÓA LUẬN TỐT NGHIỆP**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

TRẦN MẠNH DUY

XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO
VÀ ỨNG DỤNG CHỮ KÝ SỐ
KHÓA LUẬN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

Cán bộ hướng dẫn: TS. GVC. Lê Phê Đô

HÀ NỘI - 2026

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp *Xây dựng đường cong Elliptic bậc siêu cao và ứng dụng Chữ ký số* là do tôi tự tìm hiểu, nghiên cứu và phát triển dưới sự hướng dẫn của TS. GVS. Lê Phê Đô, không sao chép hay sử dụng bất kỳ phần nào từ các công trình nghiên cứu của người khác mà không trích dẫn nguồn theo quy định. Tất cả những tài liệu tham khảo đều đã được liệt kê ở phần Tài liệu tham khảo của khóa luận. Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của toàn bộ nội dung trong khóa luận này.

Tác giả khóa luận

Trần Mạnh Duy

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. GVC. Lê Phê Đô. Trong suốt thời gian qua, Thầy đã luôn dành thời gian, tận tình hướng dẫn, chỉ bảo em từng bước trong quá trình thực hiện khóa luận, giúp em định hướng đúng đắn, tháo gỡ những khó khăn và từng bước hoàn thiện đề tài nghiên cứu này.

Em cũng xin gửi lời tri ân sâu sắc đến Ban Giám hiệu cùng toàn thể quý thầy cô giáo Khoa Công nghệ thông tin, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội. Những kiến thức chuyên môn, những bài học quý báu cùng môi trường học tập, nghiên cứu tuyệt vời mà nhà trường tạo dựng trong suốt những năm tháng đại học là nền tảng vững chắc cho em trên con đường phát triển sự nghiệp sau này.

Bên cạnh đó, em xin gửi lời biết ơn vô hạn đến gia đình – điểm tựa tinh thần vững chắc và an toàn nhất của em. Cảm ơn bố mẹ và người thân đã luôn bao dung, thấu hiểu, động viên và tạo mọi điều kiện tốt nhất để em có thể hoàn toàn yên tâm theo đuổi con đường học vấn. Em cũng xin dành lời cảm ơn đến những người bạn đồng hành đã luôn kề vai sát cánh, chia sẻ những áp lực và niềm vui trong suốt thanh xuân dưới mái trường đại học.

Dù đã nỗ lực hết mình và dồn nhiều tâm huyết, nhưng do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, khóa luận chắc chắn khó tránh khỏi những thiếu sót. Em rất mong nhận được sự lượng thứ và những lời góp ý quý báu của quý thầy cô trong Hội đồng đánh giá để đề tài nghiên cứu được hoàn thiện hơn nữa.

Một lần nữa, em xin chân thành cảm ơn!

MỤC LỤC

Lời cam đoan		
Lời cảm ơn		
Mục lục		i
Tóm tắt		iv
Danh mục viết tắt		v
Danh mục mã nguồn		vii
Danh mục bảng biểu		viii
Mở đầu		1
0.1. Đóng góp của đề tài		1
0.2. Bố cục của khóa luận		2
Chương 1. Tổng quan và Giới thiệu bài toán		3
1.1. Hệ mật mã khóa công khai và Đường cong Elliptic		3
1.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã		3
1.3. Giới hạn bảo mật và nhu cầu đường cong bậc siêu cao		4
1.4. Vai trò của ngôn ngữ lập trình trong an toàn mật mã		4
1.5. Phát biểu bài toán và Phạm vi nghiên cứu		5
Chương 2. Cơ sở toán học		6
2.1. Định nghĩa Trường số nguyên tố		6
2.1.1. Cài đặt: <code>PrimeFieldConfig</code> và <code>PrimeFieldElement</code>	...	6
2.1.2. Các phép toán cơ bản		7
2.2. Đường cong elliptic Short Weierstrass		7
2.2.1. Định nghĩa		7
2.2.2. Cấu trúc nhóm		8
2.3. Phép cộng điểm (Point Addition)		8
2.3.1. Quy tắc hình học		8
2.3.2. Công thức đại số		8
2.3.3. Cài đặt		9
2.4. Phép nhân vô hướng (Scalar Multiplication)		9
2.4.1. Thuật toán Double-and-Add		9

2.4.2. Cài đặt	9
2.4.3. Tầm quan trọng	10
2.5. Tối ưu số học: Phép nhân Montgomery	10
2.5.1. Vấn đề	10
2.5.2. Biến đổi Montgomery	10
2.5.3. Chuyển đổi vào/ra không gian Montgomery	11
2.6. Lũy thừa nhanh (Square-and-Multiply)	11
Chương 3. Phương pháp xây dựng đường cong elliptic	13
3.1. Phương pháp Nhân phức	13
3.1.1. Điều kiện CM và phương trình Diophantine	13
3.1.2. Đa thức lớp Hilbert $H_D(x)$	13
3.1.3. Quy trình CM đầy đủ	14
3.1.4. Trường hợp $D = -3$: Xoắn sextic	14
3.2. Kiểm chứng chéo với BLS12-381	15
3.2.1. Mục tiêu	15
3.2.2. Đường cong BLS12-381	15
3.2.3. Quy trình kiểm chứng	16
3.2.4. Kết quả	16
Chương 4. Sơ đồ chữ ký số	18
4.1. Cặp khóa (KeyPair)	18
4.1.1. Hàm băm mở rộng	18
4.2. Chữ ký Schnorr	19
4.2.1. Lý thuyết	19
4.2.2. Cấu trúc dữ liệu	19
4.2.3. Cài đặt	20
4.2.4. Bảo mật	20
4.3. Chữ ký ECDSA	20
4.3.1. Lý thuyết	20
4.3.2. Cấu trúc dữ liệu	21
4.3.3. Cài đặt	21
4.3.4. So sánh Schnorr và ECDSA	21
4.4. Cài đặt hệ thống ký (curve1024-sig)	22

Chương 5. Cài đặt và Đánh giá	23
5.1. Đánh giá hiệu năng	23
5.1.1. Kết quả đo đạc	23
5.1.2. Phân tích: Kết quả có hợp lý về mặt lý thuyết không?	23
5.1.3. Tính khả thi trong ứng dụng thực tế	24
5.2. Đánh giá độ an toàn	25
5.2.1. An toàn trước tấn công cổ điển	25
5.2.2. An toàn trước tấn công Invalid Curve	25
5.2.3. An toàn trước tấn công MOV	25
5.2.4. An toàn trước tấn công Anomalous	26
5.2.5. Hạn chế: Timing Side-Channel Attack	26
Chương 6. Kết luận	27
6.1. Kết quả đạt được	27
6.2. Đánh giá tổng quan	27
6.2.1. So sánh với mục tiêu ban đầu	27
6.2.2. Đánh giá điểm mạnh và hạn chế	28
6.3. Hướng phát triển	28
6.3.1. Tối ưu hóa hiệu năng (ngắn hạn)	28
6.3.2. Bảo mật cấp cao (trung hạn)	29
6.3.3. Mở rộng ứng dụng (dài hạn)	29
TÀI LIỆU THAM KHẢO	30

TÓM TẮT

Khóa luận trình bày quá trình thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic (ECC) hoàn chỉnh bằng ngôn ngữ Rust, không phụ thuộc vào bất kỳ thư viện mật mã bên ngoài nào. Công trình tập trung vào hai đóng góp chính: (i) xây dựng lớp số học bignum an toàn bộ nhớ với phép nhân Montgomery và các phép toán số học cơ bản; (ii) Hệ thống được thiết kế với kiến trúc hỗ trợ trường nguyên tố tùy ý lên đến 1024-bit thông qua kiểu `U1024`. Trong quá trình phát triển và kiểm chứng, tham số BLS12-381 được sử dụng làm baseline vì đây là đường cong chuẩn đã được cộng đồng quốc tế xác minh rộng rãi. File `examples/complex_multiplication.rs` cài đặt đầy đủ quy trình CM để sinh tham số đường cong mới trên trường tùy ý. Trên nền tảng này, khóa luận cài đặt thành công hai sơ đồ chữ ký số chuẩn mực là Schnorr và ECDSA, đồng thời xây dựng bộ kiểm thử toàn diện với 36/36 test cases PASSED, chứng minh tính đúng đắn của toàn bộ hệ thống. Kết quả cho thấy khả năng tự chủ hoàn toàn trong việc phát triển hạ tầng mật mã an toàn, minh bạch và có thể kiểm chứng, đặt nền móng cho các ứng dụng mật mã thế hệ mới.

Từ khóa: mật mã đường cong elliptic, Rust, số học bignum, phép nhân Montgomery, chữ ký số Schnorr, ECDSA, bảo mật 256-bit

DANH MỤC VIẾT TẮT

Viết tắt	Tên đầy đủ
AES	Advanced Encryption Standard — Tiêu chuẩn mã hóa nâng cao
AVX	Advanced Vector Extensions — Tập lệnh vector mở rộng (Intel/AMD)
BLS	Boneh-Lynn-Shacham — Sơ đồ chữ ký tổng hợp dựa trên ghép cặp
CM	Complex Multiplication — Phương pháp Nhân phức
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator — Bộ sinh số giả ngẫu nhiên an toàn mật mã
CVE	Common Vulnerabilities and Exposures — Danh mục lỗ hổng bảo mật
DL	Discrete Logarithm — Logarit rời rạc
DLP	Discrete Logarithm Problem — Bài toán logarit rời rạc
ECC	Elliptic Curve Cryptography — Mật mã đường cong Elliptic
ECDLP	Elliptic Curve Discrete Logarithm Problem — Bài toán logarit rời rạc trên đường cong Elliptic
ECDSA	Elliptic Curve Digital Signature Algorithm — Thuật toán chữ ký số đường cong Elliptic
EUUF-CMA	Existential Unforgeability under Chosen Message Attack — Tính không thể giả mạo dưới tấn công thông điệp chọn trước
FIPS	Federal Information Processing Standards — Tiêu chuẩn xử lý thông tin liên bang (Hoa Kỳ)

Viết tắt	Tên đầy đủ
MOV	Menezes-Okamoto-Vanstone — Tấn công dùng phép ghép cặp
MSM	Multi-Scalar Multiplication — Phép nhân đa vô hướng
NAF	Non-Adjacent Form — Dạng biểu diễn không liền kề
NFS	Number Field Sieve — Thuật toán rây trường số
NIST	National Institute of Standards and Technology — Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
PKC	Public Key Cryptography — Mật mã khóa công khai
PKI	Public Key Infrastructure — Hạ tầng khóa công khai
REDC	Montgomery Reduction — Phép rút gọn Montgomery
RSA	Rivest-Shamir-Adleman — Hệ mật mã khóa công khai RSA
SHA	Secure Hash Algorithm — Thuật toán băm an toàn
SIMD	Single Instruction, Multiple Data — Một lệnh, nhiều dữ liệu
SSSA	Smart-Satoh-Araki — Tấn công đường cong dị thường (Anomalous)
SSH	Secure Shell — Giao thức đăng nhập từ xa an toàn
TLS	Transport Layer Security — Bảo mật tầng giao vận

DANH MỤC MÃ NGUỒN

2.1	Định nghĩa PrimeFieldConfig và PrimeFieldElement	6
2.2	Cài đặt phép cộng trên trường hữu hạn modulo p	7
2.3	Cài đặt phép nghịch đảo bằng Định lý nhỏ Fermat	7
2.4	Định nghĩa cấu trúc điểm AffinePoint	8
2.5	Thuật toán nhân vô hướng Double-and-Add	10
2.6	Hàm giảm bậc Montgomery (REDC)	11
2.7	Phép gán hằng-thời-gian trong lũy thừa chống tấn công kênh kề	12
4.1	Cấu trúc dữ liệu cặp khóa KeyPair	18
4.2	Cấu trúc dữ liệu chữ ký Schnorr	19
4.3	Cấu trúc dữ liệu chữ ký ECDSA	21
4.4	Giao diện dòng lệnh CLI curve1024-sig	22

DANH MỤC BẢNG BIỂU

Bảng 2.1	Vai trò của phép nhân vô hướng trong các sơ đồ mật mã ECC	10
Bảng 2.2	So sánh chi phí phép nhân thông thường và Montgomery	11
Bảng 3.1	Đa thức lớp Hilbert $H_D(x)$ với các giá trị $ D $ nhỏ	14
Bảng 3.2	Tham số đường cong BLS12-381 (giá trị hex đầy đủ tại [16])	15
Bảng 4.1	So sánh sơ đồ chữ ký Schnorr và ECDSA	21
Bảng 5.1	Kết quả đo hiệu năng hệ thống Curve1024	23
Bảng 6.1	Đối chiếu kết quả với mục tiêu đề ra	28

MỞ ĐẦU

Hệ mật mã Đường cong Elliptic (ECC) là nền tảng bảo mật của hạ tầng số hiện đại, từ giao thức TLS, SSH cho đến các hệ thống blockchain [1, 2]. Tuy nhiên, sự phụ thuộc vào các bộ tham số sinh sẵn bởi các tổ chức quốc tế đặt ra những thách thức nghiêm trọng về tự chủ công nghệ và an ninh quốc gia. Đồng thời, hầu hết các thư viện mật mã truyền thống được viết bằng C/C++ — một ngôn ngữ tiềm ẩn nhiều rủi ro về an toàn bộ nhớ.

Khóa luận này đặt mục tiêu thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic sử dụng ngôn ngữ lập trình Rust. Hệ thống được thiết kế với kiến trúc hỗ trợ trường nguyên tố tùy ý lên đến 1024-bit thông qua kiểu `U1024`. Trong quá trình phát triển và kiểm chứng, tham số BLS12-381 được sử dụng làm baseline vì đây là đường cong chuẩn đã được cộng đồng quốc tế xác minh rộng rãi. File `examples/complex_multiplication.rs` cài đặt đầy đủ quy trình CM để sinh tham số đường cong mới trên trường tùy ý. Hệ thống bao gồm toàn bộ ngăn xếp — từ lớp số nguyên lớn `U1024`, số học trường hữu hạn Montgomery, phép toán nhóm trên đường cong, đến các sơ đồ chữ ký số Schnorr và ECDSA — không phụ thuộc bất kỳ thư viện mật mã bên ngoài nào.

0.1. Đóng góp của đề tài

Khóa luận đạt được các đóng góp chính sau:

1. **Xây dựng hệ thống số học bignum hoàn chỉnh** — Tự định nghĩa kiểu dữ liệu `U1024` với phép nhân Montgomery, lũy thừa hằng-thời-gian, và nghịch đảo Fermat, đảm bảo an toàn bộ nhớ tuyệt đối nhờ kiến trúc Rust.
2. **Xây dựng đường cong elliptic bằng phương pháp Nhân phức (CM)** — Cài đặt phương pháp CM trên không gian trường tùy ý và sử dụng BLS12-381 làm baseline mặc định để kiểm chứng tính đúng đắn thư viện.
3. **Cài đặt hai sơ đồ chữ ký số** — Schnorr và ECDSA, với bộ kiểm thử bao phủ từ số học cấp thấp đến mô phỏng tấn công MOV và Anomalous.

0.2. Bố cục của khóa luận

- **Chương 1. Tổng quan và Giới thiệu bài toán:** Bối cảnh lịch sử của ECC, thực trạng các tiêu chuẩn hiện hành, vấn đề tự chủ mật mã, vai trò của ngôn ngữ lập trình, và phát biểu bài toán.
- **Chương 2. Cơ sở toán học:** Trường hữu hạn, đường cong elliptic Short Weierstrass, phép cộng điểm, nhân vô hướng, và phép nhân Montgomery.
- **Chương 3. Phương pháp xây dựng đường cong elliptic:** Phương pháp Nhân phức (CM) và kiểm chứng chéo với BLS12-381.
- **Chương 4. Sơ đồ chữ ký số:** Cấu trúc cặp khóa, Schnorr, ECDSA, và chữ ký tổng hợp BLS.
- **Chương 5. Cài đặt và Đánh giá:** Benchmark hiệu năng, phân tích an toàn trước 5 loại tấn công, và định hướng phát triển.

CHƯƠNG 1

TỔNG QUAN VÀ GIỚI THIỆU BÀI TOÁN

Chương này cung cấp cái nhìn tổng quan về bối cảnh của Hệ mật mã khóa công khai, thực trạng của các tiêu chuẩn Đường cong Elliptic hiện hành, và phân tích những rủi ro bảo mật cốt lõi về cả mặt toán học lẫn triển khai phần mềm. Từ đó, chương làm nổi bật tính cấp thiết của bài toán xây dựng đường cong Elliptic bậc siêu cao bằng ngôn ngữ an toàn bộ nhớ.

1.1. Hệ mật mã khóa công khai và Đường cong Elliptic

Sự ra đời của Mật mã khóa công khai (Public Key Cryptography - PKC), đánh dấu bởi công trình của Diffie và Hellman năm 1976 [1], là một bước ngoặt trong lịch sử an toàn thông tin. Mô hình mã hóa sử dụng khóa bất đối xứng — một khóa công khai để mã hóa/xác minh và một khóa riêng tư để giải mã/ký số — đã giải quyết triệt để bài toán phân phối khóa qua kênh truyền không an toàn mà hệ mật mã khóa đối xứng truyền thống không thể làm được.

Tuy nhiên, các hệ mật mã khóa công khai thế hệ đầu như RSA đòi hỏi kích thước khóa rất lớn để đạt mức bảo mật cao. Để đạt mức 128-bit bảo mật đối xứng (mức tối thiểu được khuyến cáo hiện nay theo NIST SP 800-57 [3]), hệ thống RSA cần sử dụng khóa dài tới 3072-bit. Nhu cầu tối ưu hóa này dẫn tới sự ra đời của Hệ mật mã Đường cong Elliptic (Elliptic Curve Cryptography - ECC), được đề xuất độc lập bởi Koblitz [4] và Miller [5] vào giữa thập niên 1980. Sức mạnh vượt trội của ECC nằm ở tỷ lệ giữa kích thước khóa và độ an toàn: ECC chỉ yêu cầu khóa 256-bit để đạt mức bảo mật 128-bit tương đương RSA 3072-bit. Kích thước khóa nhỏ hơn đáng kể giúp ECC tiêu thụ ít tài nguyên tính toán, băng thông và điện năng hơn, biến nó trở thành tiêu chuẩn vàng cho các giao thức mạng hiện đại như TLS, mạng tiền điện tử (Bitcoin, Ethereum), và các hệ thống chữ ký số điện tử [2].

1.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã

Mặc dù ECC mang lại lợi ích lớn về hiệu năng, việc triển khai ECC trong thực tế bộc lộ một vấn đề nghiêm trọng về chủ quyền công nghệ. Hiện nay, hầu hết các hệ thống phần mềm đều tái sử dụng các đường cong được định nghĩa và sinh sẵn bởi các tổ chức

quốc tế, điển hình như Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST, ví dụ tập tham số P-256) hoặc Tổ chức SECG (ví dụ secp256k1).

Việc sử dụng các hằng số toán học (các hệ số p, a, b) được sinh ra từ các “hạt giống” (seed) không giải thích được rõ ràng tiềm ẩn nguy cơ bị cài cắm “cửa hậu” (backdoor). Nguy cơ này không chỉ nằm trên lý thuyết mà đã trở thành hiện thực với sự kiện cơ quan tình báo NSA bị cáo buộc can thiệp và cài backdoor vào tiêu chuẩn thuật toán sinh số ngẫu nhiên Dual_EC_DRBG [6]. Sự kiện này đặt ra yêu cầu bức thiết về năng lực **“Tự chủ mật mã”** — nơi các hệ thống quốc gia có khả năng tự thiết kế, đánh giá và làm chủ các tham số đường cong bằng các phương pháp toán học minh bạch, không phụ thuộc vào các thư viện “hộp đen” từ nước ngoài.

1.3. Giới hạn bảo mật và nhu cầu đường cong bậc siêu cao

Xét về giới hạn an toàn trước máy tính cổ điển: khóa ECC 256-bit dù vẫn an toàn ở thời điểm hiện tại nhưng chỉ đạt biên độ an toàn 128-bit đối xứng. Thuật toán tấn công hiệu quả nhất là Pollard’s ρ [7] với độ phức tạp $O(\sqrt{r})$, trong đó r là bậc nhóm con nguyên tố.

Khóa luận này thiết kế kiến trúc lõi hỗ trợ tính toán trên trường nguyên tố lên đến **1024-bit**. Với bậc nhóm $r \approx 2^{512}$, hệ thống có năng lực cung cấp mức bảo mật xấp xỉ **256-bit đối xứng** — tương đương AES-256 và vượt xa chuẩn tối thiểu NIST. Tại mức này, thuật toán Pollard’s ρ cần $O(2^{256})$ phép toán nhóm, khiến mọi cuộc tấn công cổ điển trở thành bất khả thi. Mặc dù cấu hình mặc định sử dụng tham số BLS12-381 để kiểm thử, kiến trúc U1024 bên dưới đảm bảo khả năng mở rộng trực tiếp lên các không gian cực hạn này.

Đây là sự đánh đổi có chủ ý: hiệu năng chậm hơn đáng kể so với đường cong 256-bit nhưng đổi lại biên an toàn vượt trội, phù hợp với kịch bản bảo mật dài hạn và các ứng dụng ký ngoại tuyến.

1.4. Vai trò của ngôn ngữ lập trình trong an toàn mật mã

Trong lịch sử ngành an toàn thông tin, những sự cố bảo mật nghiêm trọng nhất của các thư viện mật mã lớn thường không xuất phát từ sai sót trong toán học lý thuyết, mà bắt nguồn từ lỗi hỏng quản lý bộ nhớ của ngôn ngữ C/C++. Các lỗi tràn bộ đệm (buffer overflow), truy cập vùng nhớ đã giải phóng (use-after-free), hay đọc quá giới hạn vùng nhớ đã gây ra hàng loạt thảm họa bảo mật, nổi tiếng nhất là sự cố Heartbleed năm 2014 trong thư viện OpenSSL [8]. Theo thống kê từ Microsoft, khoảng 70% các bản vá lỗi

bảo mật cốt lõi của họ có nguyên nhân trực tiếp từ nhóm lỗi an toàn bộ nhớ [9].

Thực tế triển khai hệ thống xử lý số nguyên lớn (BigInt) kích thước 1024-bit bằng C/C++ là thách thức cực đoan về quản lý bộ nhớ. Khóa luận này khắc phục điểm yếu kiến trúc đó bằng ngôn ngữ lập trình hệ thống **Rust**. Với cơ chế Ownership (quyền sở hữu) và Borrow Checker (kiểm tra mượn tĩnh) tại thời điểm biên dịch, trình biên dịch Rust loại bỏ hoàn toàn các lỗi an toàn bộ nhớ mà không cần bộ thu gom rác (Garbage Collector). Cách tiếp cận này giúp thư viện mật mã đạt tốc độ thực thi ngang ngửa C/C++ trong khi được bảo hành tuyệt đối về an toàn bộ nhớ.

1.5. Phát biểu bài toán và Phạm vi nghiên cứu

Dựa trên những phân tích trên, khóa luận đặt ra mục tiêu thiết kế và cài đặt từ đầu (from scratch) một thư viện mã nguồn mở cho Hệ mật mã Đường cong Elliptic với các đặc điểm:

1. **Về mặt toán học:** Cài đặt phương pháp Nhân phức (CM) trên không gian số nguyên lớn, hỗ trợ sinh tham số đường cong tùy ý. Sử dụng đường cong chuẩn BLS12-381 làm baseline để kiểm chứng.
2. **Về mặt kiến trúc phần mềm:** Mã nguồn phân tầng theo ba lớp biệt lập:
 - *Lớp Số nguyên lớn* (U1024): Kiểu dữ liệu 1024-bit với phép toán tối ưu phần cứng.
 - *Lớp Trường hữu hạn* (PrimeField): Số học modulo Montgomery bảo mật.
 - *Lớp Tọa độ điểm* (AffinePoint): Biểu diễn hình học trên đường cong.
3. **Về mặt ứng dụng:** Cài đặt, kiểm thử và đánh giá hiệu năng hai sơ đồ chữ ký số tiêu chuẩn (ECDSA [10] và Schnorr [11]).
4. **Về mặt công nghệ:** Toàn bộ hệ thống viết bằng Rust, đảm bảo an toàn bộ nhớ tuyệt đối và hiệu năng ở cấp ngôn ngữ hệ thống.

CHƯƠNG 2

CƠ SỞ TOÁN HỌC

Chương này trình bày các khái niệm toán học nền tảng được sử dụng trong toàn bộ luận văn: cấu trúc trường hữu hạn, đường cong elliptic Short Weierstrass và các phép toán nhóm trên nó [12, 13], cùng với các kỹ thuật số học tối ưu (Montgomery, Fermat) được cài đặt trực tiếp trong thư viện `curve1024`.

2.1. Định nghĩa Trường số nguyên tố

Trường số nguyên tố $\mathbb{F}_p$ là tập $\{0, 1, \dots, p-1\}$ với phép cộng và nhân lấy modulo p , trong đó p là số nguyên tố. Đây là trường hữu hạn nhỏ nhất có đặc số p .

Các tính chất quan trọng:

- **Đóng kín:** kết quả của mọi phép toán đều thuộc $\mathbb{F}_p$.
- **Nghịch đảo nhân:** mọi phần tử $a \neq 0$ có nghịch đảo a^{-1} sao cho $a \cdot a^{-1} \equiv 1 \pmod{p}$.
- **Theo Định lý nhỏ Fermat:** $a^{p-1} \equiv 1 \pmod{p}$ với mọi $a \neq 0$, do đó $a^{-1} = a^{p-2} \pmod{p}$.

2.1.1. Cài đặt: *PrimeFieldConfig* và *PrimeFieldElement*

Trong thư viện `curve1024`, trường $\mathbb{F}_p$ được biểu diễn qua trait `PrimeFieldConfig` (mang các hằng số tĩnh) và struct `PrimeFieldElement<C>` (phần tử trường):

```
pub trait PrimeFieldConfig {
    const MODULUS: U1024;    // p: số nguyên tố trường
    const R2: U1024;         // R^2 mod p, dùng cho Montgomery
    const N_PRIME: U1024;    // -p^{-1} mod 2^{1024}, dùng cho REDC
}

pub struct PrimeFieldElement<C: PrimeFieldConfig> {
    value: U1024,            // biểu diễn Montgomery: a.R mod p
}
```

Mã nguồn 2.1. Định nghĩa PrimeFieldConfig và PrimeFieldElement

Thiết kế này cho phép trình biên dịch mở inline toàn bộ hằng số tại thời gian biên dịch (zero-cost abstraction): hai trường khác nhau $\mathbb{F}_p$ và $\mathbb{F}_r$ dùng chung code nhưng có tham số riêng biệt không thể nhầm lẫn.

2.1.2. Các phép toán cơ bản

Cộng và trừ thực hiện bằng cộng/trừ số nguyên rồi hiệu chỉnh modulo:

```
// Cộng: (a + b) mod p
fn add(self, rhs: Self) -> Self {
    let (sum, carry) = self.value.carrying_add(&rhs.value);
    let (sub, borrow) = sum.borrowing_sub(&C::MODULUS);
    // Chọn kết quả không cần phép chia
    conditional_select(sum, sub, carry || !borrow)
}
```

Mã nguồn 2.2. Cài đặt phép cộng trên trường hữu hạn modulo p

Phép cộng không cần phép chia đầy đủ — chỉ cần một phép trừ có điều kiện. Chi phí: $O(n)$ với n là số limb (16 limb x 64 bit = 1024 bit).

Nhân dùng phép nhân Montgomery (xem §2.4).

Nghịch đảo dùng lũy thừa Fermat: $a^{-1} = a^{p-2} \pmod{p}$, thực hiện bởi hàm `pow` với phép nhân bình phương liên tiếp (square-and-multiply):

```
pub fn inv(&self) -> Self {
    self.pow(C::MODULUS - 2) // a^(p-2) mod p
}
```

Mã nguồn 2.3. Cài đặt phép nghịch đảo bằng Định lý nhỏ Fermat

2.2. Đường cong elliptic Short Weierstrass

2.2.1. Định nghĩa

Trên $\mathbb{F}_p$ với $p > 3$, **đường cong elliptic dạng Short Weierstrass** là tập hợp:

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p \times \mathbb{F}_p \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

trong đó $\mathcal{O}$ là **điểm vô cực** (point at infinity) — phần tử đặc biệt không thuộc mặt phẳng affine, đóng vai trò phần tử đơn vị của nhóm.

Điều kiện không suy biến: Biệt thức $\Delta = -16(4a^3 + 27b^2) \neq 0$, đảm bảo đường cong không có điểm kỳ dị (cusp hay node). Kiểm tra này được thực hiện ngầm qua quá trình CM: với $D = -3$, luôn có $a = 0$ và điều kiện thu gọn thành $b \neq 0$.

2.2.2. Cấu trúc nhóm

Tập $E(\mathbb{F}_p)$ tạo thành một **nhóm Abel hữu hạn** với phép cộng điểm. Bậc của nhóm $\#E(\mathbb{F}_p) = p + 1 - t$ (định lý Hasse [12]), trong đó $|t| \leq 2\sqrt{p}$ là vết Frobenius.

Trong cài đặt:

```
pub struct AffinePoint<C: SWCurveConfig> {
    pub x: FieldElement<C::BaseField>,
    pub y: FieldElement<C::BaseField>,
    pub is_infinite: bool,
}
```

Mã nguồn 2.4. Định nghĩa cấu trúc điểm AffinePoint

2.3. Phép cộng điểm (Point Addition)

2.3.1. Quy tắc hình học

Phép cộng trên E được định nghĩa bằng quy tắc “dây cung và tiếp tuyến”:

- **Phần tử đơn vị:** $P + \mathcal{O} = \mathcal{O} + P = P$ với mọi P .
- **Phần tử nghịch đảo:** $-(x, y) = (x, -y)$, vì $P + (-P) = \mathcal{O}$.
- **Cộng hai điểm phân biệt** $P_1 \neq \pm P_2$: vẽ đường thẳng qua P_1, P_2 , điểm giao thứ ba với đường cong là R' , phản chiếu qua trục x cho $P_3 = P_1 + P_2 = -R'$.
- **Nhân đôi** $P + P = 2P$: dùng đường tiếp tuyến tại P .

2.3.2. Công thức đại số

Cho $P_1 = (x_1, y_1)$ và $P_2 = (x_2, y_2)$ trên $y^2 = x^3 + ax + b$, điểm tổng $P_3 = (x_3, y_3)$:

Cộng hai điểm phân biệt ($P_1 \neq P_2$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Nhân đôi ($P_1 = P_2$):

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Mỗi công thức cần **1 phép nghịch đảo** (tính λ), 2–3 phép nhân, và 4–6 phép cộng/trừ trong $\mathbb{F}_p$.

2.3.3. Cài đặt

Các bước cộng và nhân đôi điểm trên đường cong được cài đặt trực tiếp từ công thức đại số, thông qua hai hàm `add()` và `double()`. Đặc biệt, hệ thống tích hợp sẵn hàm `is_on_curve()` trong quá trình khởi tạo điểm kết quả thông qua hàm `new()` để kiểm tra và loại bỏ ngay lập tức các điểm không hợp lệ. Điều này giúp hệ thống bắt lỗi sớm và miễn nhiễm hoàn toàn với tấn công đường cong không hợp lệ (Invalid Curve Attack).

Chi tiết mã nguồn cài đặt các hàm `add()` và `double()` được trình bày tại mã nguồn công khai của dự án [14].

2.4. Phép nhân vô hướng (Scalar Multiplication)

2.4.1. Thuật toán Double-and-Add

Phép nhân vô hướng $[k]P = P + P + \dots + P$ (k lần) được tính hiệu quả bằng **Double-and-Add** (tương tự binary exponentiation), với độ phức tạp $O(\log k)$ thay vì $O(k)$:

Input: $P \in E(F_p)$, $k \in [0, r-1]$

Output: $[k]P$

```
R ← O (điểm vô cực)
B ← P
for i = 0 to 1023:
    if bit i của k bằng 1:
        R ← R + B
    B ← 2B
return R
```

Vòng lặp luôn chạy đủ 1024 bước do kiểu `U1024` duyệt toàn bộ 1024 bit. Với BLS12-381 ($r \approx 2^{255}$), 769 vòng đầu luôn gập bit 0 và chỉ thực hiện `double`, không thực hiện `add`. Khi tích hợp tham số 1024-bit thực thụ ($r \approx 2^{512}$), toàn bộ 1024 vòng đều có bit có nghĩa, trung bình 512 lần `add` thêm.

2.4.2. Cài đặt

```
pub fn mul(&self, scalar: &U1024) -> Self {
    let mut result = Self::infinite();
    let mut base   = *self;
    for i in 0..1024 {
```

```

    let limb_idx = i / 64;
    let bit_idx  = i % 64;
    if (scalar.0[limb_idx] >> bit_idx) & 1 == 1 {
        result = result.add(&base);
    }
    base = base.double();
}
result
}

```

Mã nguồn 2.5. Thuật toán nhân vô hướng Double-and-Add

Mỗi lần gọi `mul` thực hiện đúng 1024 lần `double` và tối đa 1024 lần `add`. Chi phí tính toán chủ yếu đến từ phép nghịch đảo trong $\mathbb{F}_p$ (mỗi lần `add/double` cần 1 nghịch đảo, tức $a^{p-2} \pmod{p}$ — một lũy thừa trên U1024).

2.4.3. Tầm quan trọng

Phép nhân vô hướng là hàm cốt lõi của mọi sơ đồ mật mã ECC:

Bảng 2.1. Vai trò của phép nhân vô hướng trong các sơ đồ mật mã ECC

Thao tác	Ký hiệu	Ứng dụng
Sinh khóa công khai	$Q = [d]G$	Từ khóa riêng d
Cam kết ký (Schnorr/ECDSA)	$R = [k]G$	Nonce commitment
Xác minh Schnorr	$[s]G$ và $[e]Q$	Hai lần nhân vô hướng
Xác minh ECDSA	$[u_1]G + [u_2]Q$	Hai lần nhân vô hướng

2.5. Tối ưu số học: Phép nhân Montgomery

2.5.1. Vấn đề

Phép nhân thông thường $a \cdot b \pmod{p}$ đòi hỏi một phép chia đầy đủ để lấy số dư — rất đắt với số 1024-bit. Montgomery đề xuất thực hiện phép nhân trong **không gian Montgomery** để tránh phép chia [15].

2.5.2. Biến đổi Montgomery

Chọn $R = 2^{1024}$ (cơ số nguyên tố cùng nhau với p). **Dạng Montgomery** của a là $\hat{a} = a \cdot R \pmod{p}$.

Phép nhân Montgomery (**REDC**) tính $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod{p}$ từ $\hat{a}$ và $\hat{b}$, chỉ dùng phép chia cho R (shift phải 1024 bit — rất rẻ) thay vì chia cho p .

Bảng 2.2. So sánh chi phí phép nhân thông thường và Montgomery

Phép tính	Thông thường	Montgomery
$a \cdot b \pmod{p}$	cần chia cho p	shift + trừ có điều kiện
Chi phí	$O(n^2)$ + chia	$O(n^2)$ không chia

Quy trình REDC (với ba hằng số biên dịch: $R^2 \pmod{p}$, $-p^{-1} \pmod{R}$):

1. Nhân $lo, hi = \hat{a} \cdot \hat{b}$
2. $m = lo \cdot N' \pmod{R}$ với $N' = -p^{-1} \pmod{R}$
3. $t = (hi, lo) + m \cdot p$ (chỉ giữ phần trên R)
4. Nếu $t \geq p$: $t \leftarrow t - p$

```
fn reduce(lo: &U1024, hi: &U1024) -> U1024 {
    let (m, _) = lo.widening_mul(&C::N_PRIME); // m = lo . N' mod R
    let (mn_lo, mn_hi) = m.widening_mul(&C::MODULUS); // m . p
    let (_, carry_lo) = lo.carrying_add(&mn_lo);
    let (t, carry_hi) = hi.carrying_add(&mn_hi);
    // Bước hiệu chỉnh cuối: trừ p nếu cần
    let (sub, borrow) = t.borrowing_sub(&C::MODULUS);
    if carry_hi || carry_lo { sub } else if !borrow { sub } else { t }
}
```

Mã nguồn 2.6. Hàm giảm bậc Montgomery (REDC)

2.5.3. Chuyển đổi vào/ra không gian Montgomery

- **Chuyển vào:** `PrimeFieldElement::new(a)` tính $a \cdot R^2 \cdot R^{-1} = a \cdot R \pmod{p}$.
- **Chuyển ra:** `to_u1024()` gọi `reduce(value, 0) = \hat{a} \cdot R^{-1} = a \pmod{p}`.

Trong thực tế, hầu hết tính toán diễn ra trong không gian Montgomery — chỉ chuyển ra khi cần so sánh hay xuất kết quả, giúp loại bỏ hoàn toàn phép chia trong vòng lặp tính toán chính.

2.6. Lũy thừa nhanh (Square-and-Multiply)

Hàm `pow` thực hiện lũy thừa $a^e \pmod{p}$ bằng phương pháp **square-and-multiply** (nhân bình phương liên tiếp). Việc cài đặt cấp thấp tiềm ẩn rủi ro lộ lọt thông tin khóa qua độ chênh lệch thời gian thực thi của các khối lệnh điều kiện.

Để bảo vệ hệ thống trước tấn công kênh kề (timing side-channel), thay vì sử dụng cấu trúc rẽ nhánh `if-else` (mở ra nguy cơ phán đoán bit số mũ dựa trên thời gian thực thi của CPU branch-prediction), thủ tục sử dụng cách tính chọn hằng-thời-gian (constant-time execution):

```
// Cả hai nhánh điều kiện đều được tính toán  
let product = res * base;  
// Chỉ định phép gán qua hàm chuyển đổi bit, không dùng rẽ nhánh điều khiển  
res = Self::conditional_select(&res, &product, bit.into());
```

Mã nguồn 2.7. Phép gán hằng-thời-gian trong lũy thừa chống tấn công kênh kề

Hàm `conditional_select` đảm bảo cả hai nhánh (khi bit bằng 0 hoặc 1) đều tiêu tốn một lượng chu kỳ lệnh (clock cycles) CPU bằng nhau hoàn toàn. Chi tiết mã nguồn vòng lặp 1024-bit của hàm `pow` được đính kèm tại mã nguồn công khai của dự án [14]. Nghịch đảo $a^{-1} = a^{p-2}$ dùng `pow` với $e = p - 2$: 1024 lần bình phương và ~512 lần nhân (trung bình), tổng ≈ 1536 phép nhân Montgomery.

CHƯƠNG 3

PHƯƠNG PHÁP XÂY DỰNG ĐƯỜNG CONG ELLIPTIC

Chương này trình bày phương pháp xây dựng đường cong elliptic có bậc nhóm xác định bằng Phương pháp Nhân phức (CM) và Đa thức lớp Hilbert [12]. Tiếp đó, chương trình bày quy trình kiểm chứng chéo (cross-validation) tính đúng đắn của thư viện bằng đường cong chuẩn BLS12-381 [16].

3.1. Phương pháp Nhân phức

Phương pháp Nhân phức (CM) là công cụ nền tảng để **xây dựng đường cong elliptic có số điểm định trước**. Thay vì chọn ngẫu nhiên các hệ số a, b rồi đếm điểm, CM đi ngược lại: bắt đầu từ bậc nhóm mong muốn $\#E(\mathbb{F}_p) = p + 1 - t$ rồi suy ngược ra phương trình đường cong.

Ý tưởng xuất phát từ lý thuyết số học phức: mỗi đường cong elliptic E sở hữu một **vành nội cấu** (endomorphism ring) $\text{End}(E)$. Với đường cong thông thường (không siêu dị), vành này đẳng cấu với một **thứ tự trong trường số** $\mathbb{Q}(\sqrt{D})$, trong đó $D < 0$ là số nguyên âm gọi là **biệt thức CM**. Giá trị D đặc trưng hoàn toàn cho “loại” đường cong về mặt số học phức.

3.1.1. Điều kiện CM và phương trình Diophantine

Để tồn tại một đường cong elliptic trên $\mathbb{F}_p$ có vết Frobenius t và biệt thức CM là D , điều kiện cần và đủ là phương trình Diophantine sau phải có nghiệm nguyên:

$$4p = t^2 + |D| \cdot v^2$$

trong đó p là đặc số trường, t là vết Frobenius, v là một số nguyên dương, và $D < 0$. Phương trình này **ràng buộc trực tiếp** bộ tham số (p, t, D) với nhau; không phải mọi bộ ba nào cũng có thể thỏa mãn đồng thời.

Ví dụ: Với $D = -3$ (biệt thức của secp256k1, BLS12-381), phương trình trở thành $4p = t^2 + 3v^2$.

3.1.2. Đa thức lớp Hilbert $H_D(x)$

Khi đã có D , bước tiếp theo là tính **Đa thức lớp Hilbert** (Hilbert class polynomial) $H_D(x)$. Đây là đa thức hệ số nguyên được xây dựng từ lý thuyết số học phức mà nghiệm

phức của nó chính là các **j -invariant** của tất cả đường cong elliptic phức $\mathbb{C}/\mathcal{O}$ với thứ tự CM là $\mathcal{O}$.

Bậc của $H_D(x)$ bằng **số lớp** (class number) $h(D)$ — một bất biến số học của thứ tự $\mathbb{Z}[\sqrt{D}]$. Với $|D|$ nhỏ:

Bảng 3.1. Đa thức lớp Hilbert $H_D(x)$ với các giá trị $|D|$ nhỏ

$ D $	$h(D)$	$H_D(x)$
3	1	x
4	1	$x - 1728$
7	1	$x + 3375$
11	1	$x + 32768$
23	3	$x^3 + \dots$

Với $|D| = 3$ và $|D| = 4$, đa thức này bậc 1, nên **j -invariant được xác định ngay lập tức**: $j = 0$ và $j = 1728$ tương ứng. Đây là lý do tại sao secp256k1 ($j = 0$, $D = -3$) và nhiều đường cong BLS có dạng đơn giản $y^2 = x^3 + b$.

3.1.3. Quy trình CM đầy đủ

Cho bộ tham số (p, t, D) thỏa mãn $4p = t^2 + |D|v^2$:

1. **Tính j -invariant**: Tìm nghiệm $j_0 \in \mathbb{F}_p$ của $H_D(x) \pmod{p}$.
2. **Suy ra hệ số đường cong**: Từ j_0 , tính $c = j_0 \cdot (1728 - j_0)^{-1} \pmod{p}$, sau đó:
 - $a = 3c, b = 2c$ (trường hợp tổng quát $j_0 \neq 0, 1728$)
 - $a = 0, b$ tùy ý (khi $j_0 = 0$, tức $D = -3$)
 - a tùy ý, $b = 0$ (khi $j_0 = 1728$, tức $D = -4$)
3. **Kiểm tra xoắn (Twist test)**: Đường cong $E : y^2 = x^3 + ax + b$ và đường cong xoắn $E' : y^2 = x^3 + a\delta^2x + b\delta^3$ (với δ là phần tử không phải thặng dư bậc hai) có tổng số điểm $\#E + \#E' = 2(p + 1)$. Ta cần chọn đúng đường cong có $r \mid \#E$.
4. **Tìm điểm sinh (Generator)**: Nhân ngẫu nhiên một điểm trên đường cong với hệ số cofactor $h = \#E/r$ để tìm điểm sinh G có bậc chính xác là r .

3.1.4. Trường hợp $D = -3$: Xoắn sextic

Với $D = -3$, tình huống phức tạp hơn vì đường cong $y^2 = x^3 + b$ có **6 xoắn** (sextic twists) thay vì chỉ 2. Sáu vết Frobenius khả dĩ là:

$$t, \quad -t, \quad \frac{t \pm 3v}{2}, \quad \frac{-t \pm 3v}{2}$$

trong đó $4p = t^2 + 3v^2$. Để xác định xoắn đúng, ta lần lượt thử các giá trị b nhỏ và kiểm tra số điểm thực sự của đường cong $y^2 = x^3 + b$ bằng cách tính phép nhân vô hướng $[p + 1 - t']P$ với điểm ngẫu nhiên P .

Trong cài đặt của luận văn, hàm `find_twist` thực hiện tính toán và lọc nghiệm từ 6 vết Frobenius này. Chi tiết mã nguồn tính toán mảng ứng viên được trình bày tại mã nguồn công khai của dự án [14].

3.2. Kiểm chứng chéo với BLS12-381

3.2.1. Mục tiêu

Thư viện sử dụng tham số BLS12-381 làm cấu hình mặc định trong quá trình phát triển và kiểm thử, vì hai lý do sau: (i) BLS12-381 là đường cong chuẩn công nghiệp đã được kiểm chứng độc lập, cho phép so sánh kết quả dễ dàng; (ii) quy trình CM sinh tham số 1024-bit thực thụ đã được cài đặt trong `examples/complex_multiplication.rs` nhưng chưa được tích hợp vào config mặc định do giới hạn thời gian tính toán.

Ý tưởng cốt lõi: nếu cùng một mã nguồn thư viện lõi (cùng kiểu U1024, cùng hàm `montgomery_mul`, cùng `scalar_mul`, cùng `Schnorr::sign/verify` và `ECDSA::sign/verify`) hoạt động đúng trên đường cong chuẩn công nghiệp đã được kiểm chứng rộng rãi, thì có thể kết luận rằng thư viện đúng đắn **độc lập với bất kỳ bộ tham số cụ thể nào**. Kiến trúc U1024 cho phép dễ dàng chuyển đổi sang tham số mới bằng cách cập nhật cấu hình `curve1024.toml`.

3.2.2. Đường cong BLS12-381

BLS12-381 [16] là đường cong elliptic được sử dụng rộng rãi trong Ethereum 2.0, Zcash, và nhiều hệ thống ZK-proof. Các tham số chính:

Bảng 3.2. Tham số đường cong BLS12-381 (giá trị hex đầy đủ tại [16])

Tham số	Giá trị
Trường cơ sở p	381-bit (số nguyên tố)
Bậc nhóm r	255-bit (số nguyên tố)
Hệ số a	0
Hệ số b	4

Tham số	Giá trị
Biệt thức CM	$D = -3$
Bậc nhúng k	12

Thư viện tham chiếu `zkcrypto/bls12_381` cung cấp các giá trị đã được kiểm chứng độc lập bởi cộng đồng mật mã quốc tế.

3.2.3. Quy trình kiểm chứng

Quy trình kiểm chứng chéo thực hiện các bước sau:

1. **Sử dụng tham số BLS12-381** làm cấu hình chính xác cho quá trình biên dịch và khởi chạy hệ thống.
2. **Chạy toàn bộ bộ kiểm thử** với tham số BLS12-381, bao gồm:
 - Test số học cấp thấp: phép cộng, trừ, nhân, lũy thừa $U1024$
 - Test trường hữu hạn: phép nhân Montgomery, nghịch đảo Fermat
 - Test đường cong: phép cộng điểm, nhân vô hướng, kiểm tra điểm trên đường cong
 - Test chữ ký: ký và xác minh Schnorr, ký và xác minh ECDSA
 - Test an toàn: kháng tấn công MOV, kháng tấn công Anomalous, kháng Invalid Curve
3. **So sánh kết quả** với thư viện tham chiếu `zkcrypto/bls12_381` để đảm bảo tính nhất quán.

3.2.4. Kết quả

Toàn bộ bộ kiểm thử chạy với tham số BLS12-381 cho kết quả **36/36 test PASSED, 0 failed**. Điều này chứng minh:

- **Tính đúng đắn của lớp số học:** Phép nhân Montgomery, lũy thừa, nghịch đảo hoạt động chính xác cho trường 381-bit trên nền cấu trúc 1024-bit ($U1024$).
- **Tính đúng đắn của phép toán đường cong:** Phép cộng điểm, nhân vô hướng, và tìm điểm sinh đều cho kết quả đúng đắn.
- **Tính đúng đắn của sơ đồ chữ ký:** Schnorr và ECDSA ký/xác minh chính xác.
- **Tính tổng quát của thư viện:** Mã nguồn không bị ràng buộc vào bất kỳ đường cong cụ thể nào — chỉ cần thay đổi file cấu hình TOML là có thể vận hành trên đường cong mới.

Kết quả này là bằng chứng mạnh mẽ nhất cho tính đúng đắn của thư viện, vì BLS12-381 đã được kiểm chứng độc lập bởi hàng trăm nhà nghiên cứu và được triển khai trong các hệ thống xử lý hàng tỷ USD giá trị.

Ghi chú về tham số hiện tại: File `config/curve1024.toml` trong phiên bản hiện tại sử dụng tham số BLS12-381 (trường 381-bit, bậc nhóm 255-bit) thay vì tham số 1024-bit mới. Quyết định này có hai lý do thực tế: (i) BLS12-381 cho phép kiểm chứng tính đúng đắn của thư viện bằng cách so sánh với test vector từ `zkcrypto/bls12_381`; (ii) việc tính toán và tích hợp tham số CM 1024-bit đầy đủ đòi hỏi thời gian tính toán đáng kể. Kiến trúc U1024 và toàn bộ ngăn xếp mật mã được thiết kế để hỗ trợ tham số tùy ý — chỉ cần thay đổi `curve1024.toml` là hệ thống hoạt động với bộ tham số mới.

CHƯƠNG 4

SƠ ĐỒ CHỮ KÝ SỐ

Chương này trình bày hai sơ đồ chữ ký số được xây dựng trên nền tảng đường cong elliptic ở Chương 3: **Schnorr** và **ECDSA**. Ngoài ra, chương trình bày lý thuyết nền tảng của **chữ ký tổng hợp BLS** như hướng phát triển. Với mỗi sơ đồ, chúng ta đi qua lý thuyết cốt lõi, phân tích cấu trúc dữ liệu và cài đặt thực tế bằng Rust. Kết quả đo hiệu năng và phân tích bảo mật được trình bày riêng tại **Chương 5**.

4.1. Cặp khóa (KeyPair)

Mọi sơ đồ chữ ký đều dùng chung cấu trúc cặp khóa:

```
pub struct KeyPair<C: SWCurveConfig> {  
    pub private_key: U1024,           //  $d$  in  $[1, r-1]$ , bí mật  
    pub public_key: AffinePoint<C>, //  $Q = [d]G$ , công khai  
}
```

Mã nguồn 4.1. Cấu trúc dữ liệu cặp khóa KeyPair

- **Khóa riêng** d : một số nguyên ngẫu nhiên trong $[1, r - 1]$, sinh bởi CSPRNG (`U1024::rand`).
- **Khóa công khai** $Q = [d]G$: tính bằng phép nhân vô hướng. Với $r \approx 2^{512}$, tính Q từ d là tính khả thi (mili giây), nhưng bài toán ngược (tính d từ Q) là ECDLP — độ phức tạp $O(2^{256})$.

Cặp khóa được lưu và đọc từ file nhị phân 128 byte (big-endian) với quyền truy cập khắt khe 0600 trên hệ điều hành Unix phân quyền để giảm rủi ro bị đọc lén bởi các tài khoản người dùng khác trên cùng máy chủ. Các thao tác khởi tạo (`generate`) ngẫu nhiên và truy xuất dữ liệu tĩnh (`save`) được xử lý bằng API tiêu chuẩn. Chi tiết mã nguồn quản lý vòng đời cặp khóa được đính kèm tại mã nguồn công khai của dự án [14].

4.1.1. Hàm băm mở rộng

Hàm `hash_message` thực hiện 4 lần SHA-256 với domain separator byte $i \in \{0, 1, 2, 3\}$ được nối (concatenate) vào trước thông điệp, sau đó ghép 4 digest 32-byte

thành một buffer 128-byte và đọc dưới dạng U1024 big-endian. Đây là cấu trúc **domain-separated hash concatenation** đơn giản nhằm tạo ra không gian đầu ra đủ rộng để tương thích với trường 1024-bit, không phải là một hàm băm chuẩn hóa mới.

Giới hạn của cách tiếp cận này là hiện chưa có chứng minh chính thức (formal proof) về khả năng kháng đụng độ (collision resistance) cho cấu trúc này so với các hàm tiêu chuẩn độ dài tùy biến như SHAKE hay BLAKE2. Việc thay thế bằng một hàm băm chuẩn mực được tối ưu hóa cho không gian lớn là một điểm cần cải thiện trong các phiên bản sau để đạt độ an toàn mật mã nghiêm ngặt. Chi tiết cài đặt thuật toán kết hợp băm được cung cấp tại mã nguồn công khai của dự án [14].

4.2. Chữ ký Schnorr

4.2.1. Lý thuyết

Schnorr là sơ đồ chữ ký Sigma-protocol dựa trên giả thuyết logarit rời rạc (DL) [11]. Nó được ưa chuộng vì độ đơn giản của chứng minh bảo mật (trong mô hình Oracle ngẫu nhiên) và hỗ trợ tổng hợp tự nhiên.

Ký: Cho khóa riêng $d \in [1, r - 1]$ và thông điệp m :

1. Chọn ngẫu nhiên $k \xleftarrow{R} [1, r - 1]$
2. Tính điểm $R = [k]G$
3. Tính thách thức $e = H(R_x || R_y || m) \pmod{r}$
4. Tính $s = k + e \cdot d \pmod{r}$
5. Chữ ký: $\sigma = (R, s)$

Xác minh: Cho khóa công khai Q và chữ ký (R, s) :

1. Tính $e = H(R_x || R_y || m)$
2. Kiểm tra $[s]G = R + [e]Q$

Tính đúng đắn: $[s]G = [k + ed]G = [k]G + [e][d]G = R + [e]Q$ [ok]

4.2.2. Cấu trúc dữ liệu

```
pub struct SchnorrSignature<C: SWCurveConfig> {  
    pub r_point: AffinePoint<C>,    // R = [k]G, 256 byte  
    pub s:      U1024,               // s in [0, r-1], 128 byte  
}
```

Mã nguồn 4.2. Cấu trúc dữ liệu chữ ký Schnorr

Kích thước chữ ký: $2 \times 128 + 128 = 384$ byte (R gồm hai tọa độ x, y mỗi tọa độ 128 byte, cộng thêm s 128 byte).

4.2.3. Cài đặt

Các bước ký và xác minh thuật toán Schnorr được nhóm tác giả cài đặt bám sát với quy trình lý thuyết toán học (đã trình bày ở Mục 4.2.1), tận dụng triệt để kiến trúc cấp phát và an toàn bộ nhớ của Rust để loại bỏ lỗi tràn giới hạn (buffer overflow) sinh ra từ việc xử lý chuỗi số lớn. Thuật toán cài đặt tích hợp hàm băm nội sinh `challenge_hash` dùng nối trực tiếp các thành phần R_x, R_y, m trước khi gọi hàm băm mở rộng. Quy trình gộp này đảm bảo “thách thức” sinh ra phụ thuộc định danh vào toàn bộ ngữ cảnh thông điệp, triệt tiêu mọi rủi ro tấn công giả mạo (forgery) bóc tách dữ liệu — đặc biệt theo kiểu công kích Pohlig-Hellman trên phần tử nonce.

Chi tiết mã nguồn của các hàm `sign()` và `verify()` thuộc thuật toán Schnorr được đính kèm tại mã nguồn công khai của dự án [14].

4.2.4. Bảo mật

- **Unforgeability (EUFCMA):** Chứng minh được trong Random Oracle Model dưới giả thuyết DL. Kẻ tấn công cần giải ECDLP hoặc bẻ hàm băm.
- **Nguy cơ nonce tái sử dụng:** Nếu k bị tái sử dụng cho hai thông điệp khác nhau, kẻ công phá có thể tính $d = (s_1 - s_2)/(e_1 - e_2) \pmod{r}$. Cài đặt dùng CSPRNG (`rand::thread_rng`) cho mỗi lần ký.

4.3. Chữ ký ECDSA

4.3.1. Lý thuyết

ECDSA (Elliptic Curve Digital Signature Algorithm) là chuẩn IEEE P1363 và FIPS 186-4 [10]. Nó khác Schnorr ở chỗ r trong chữ ký là **tọa độ x của điểm R** lấy modulo n , thay vì bản thân điểm R .

Ký: Cho khóa riêng d và thông điệp m :

1. $k \xleftarrow{R} [1, r - 1]$
2. $R = [k]G$
3. $r_{\text{sig}} = R_x \pmod{n}$ (nếu $r_{\text{sig}} = 0$, thử lại)
4. $e = H(m)$
5. $s = k^{-1}(e + r_{\text{sig}} \cdot d) \pmod{n}$ (nếu $s = 0$, thử lại)
6. Chữ ký: $\sigma = (r_{\text{sig}}, s)$

Xác minh:

1. Kiểm tra $r_{\text{sig}}, s \in [1, n - 1]$
2. $e = H(m), w = s^{-1} \pmod{n}$
3. $u_1 = ew, u_2 = r_{\text{sig}} \cdot w$, cả hai tính mod n
4. $P = [u_1]G + [u_2]Q$
5. Hợp lệ khi $P \neq \mathcal{O}$ và $P_x \pmod{n} = r_{\text{sig}}$

4.3.2. Cấu trúc dữ liệu

```
pub struct EcdsaSignature {
    pub r: U1024, // r_sig = R.x mod n, 128 byte
    pub s: U1024, // s in [1, n-1], 128 byte
}
```

Mã nguồn 4.3. Cấu trúc dữ liệu chữ ký ECDSA

Kích thước chữ ký: $128 + 128 = 256$ byte — nhỏ hơn Schnorr vì không lưu điểm R đầy đủ.

4.3.3. Cài đặt

Tương tự Schnorr, các thủ tục ký xác thực và kiểm chứng chuẩn mực của thuật toán ECDSA cũng được triển khai trong không gian số học bằng các thao tác đại số trực tiếp trên cấu trúc thành phần `FieldElement`. Tại quy trình ký sinh, tiến trình tạo phần tử ngẫu nhiên nonce được rà soát khắt khe và lập tức phải loại bỏ (chu kỳ thử lại mới) nếu tổ hợp tạo ra vi phạm hệ số tiệm cận, cụ thể rơi vào 2 trường hợp suy biến tiềm ẩn: $r_{\text{sig}} = 0$ hoặc $s = 0$. Logic bóc lỗi này tuân thủ trọn vẹn đặc tả bảo mật thuộc tiêu chuẩn chuẩn hóa FIPS 186-4 được ban hành riêng cho ECDSA.

Chi tiết mã nguồn của các hàm `sign()` và `verify()` thuộc thuật toán ECDSA được đính kèm tại mã nguồn công khai của dự án [14].

4.3.4. So sánh Schnorr và ECDSA

Bảng 4.1. So sánh sơ đồ chữ ký Schnorr và ECDSA

Tiêu chí	Schnorr	ECDSA
Chuẩn hóa	ISO/IEC 14888-3	FIPS 186-4, IEEE P1363
Kích thước chữ ký	384 byte ($2 \times 128 + 128$)	256 byte (2×128)
Phép nhân vô hướng ký	1	1

Tiêu chí	Schnorr	ECDSA
Phép nhân vô hướng xác minh	2 (tuần tự)	2 (tuần tự)
Bảo mật chứng minh	RO Model, DL	Heuristic, DL + hash
Hỗ trợ tổng hợp	Có (MuSig, FROST)	Không
Rủi ro nonce tái sử dụng	Lộ khóa riêng	Lộ khóa riêng

4.4. Cài đặt hệ thống ký (**curve1024-sig**)

Binary `curve1024-sig` cung cấp giao diện dòng lệnh kiểu GPG cho toàn bộ vòng đời khóa:

```
curve1024-sig keygen [-o <file>]
curve1024-sig sign -f <file> [-k <key>] [--scheme schnorr|ecdsa]
curve1024-sig verify -f <file> [-k <pub>]
curve1024-sig export-pub [-k <priv>] [-o <pub>]
```

Mã nguồn 4.4. Giao diện dòng lệnh CLI `curve1024-sig`

Các tham số được đọc từ `config/curve1024.toml` thông qua `build.rs` để sinh hằng số tĩnh — đảm bảo zero-cost abstraction và không có chi phí khởi tạo runtime.

CHƯƠNG 5

CÀI ĐẶT VÀ ĐÁNH GIÁ

Chương này đánh giá toàn diện hệ thống đã xây dựng, không chỉ dừng lại ở trình bày số liệu mà tập trung **chứng minh giá trị** của hệ thống trên hai phương diện: hiệu năng thực tế và độ an toàn mật mã. Mục tiêu trả lời câu hỏi: *Các con số đo được nói lên điều gì? Hệ thống này có an toàn không? Có dùng được trong thực tế không?*

5.1. Đánh giá hiệu năng

5.1.1. Kết quả đo đạc

Toàn bộ hệ thống được đo bằng framework Criterion (release build, tối ưu LLVM O3, 10 mẫu/phép đo, phần cứng thông thường không có AVX-512). Kết quả cụ thể như sau:

Bảng 5.1. Kết quả đo hiệu năng hệ thống Curve1024

Phép đo	Min	Mean	Max
KeyPair::generate	1.629 s	1.687 s	1.757 s
Schnorr/sign	1.473 s	1.551 s	1.641 s
Schnorr/verify	2.656 s	2.831 s	3.025 s
ECDSA/sign	1.524 s	1.606 s	1.711 s
ECDSA/verify	2.847 s	3.140 s	3.459 s
Nhân vô hướng $[k]G$	1.061 s	~1.1–1.3 s	1.489 s

5.1.2. Phân tích: Kết quả có hợp lý về mặt lý thuyết không?

Trước tiên cần nhìn nhận rằng hệ thống hiện tại chạy trên tham số BLS12-381 ($p = 381$ -bit) với kiến trúc U1024 1024-bit. Do U1024 luôn xử lý đủ 16 limb (mỗi limb 64-bit) bất kể kích thước thực tế của p , chi phí tính toán tương đương với trường 1024-bit đầy đủ. Đây là nguyên nhân chính dẫn đến thời gian ~ 1.5 s/phép ký, và cũng là minh chứng trực tiếp cho khả năng mở rộng của kiến trúc khi tích hợp tham số 1024-bit thực thụ.

Phép nhân vô hướng dùng thuật toán Double-and-Add với **1024 vòng lặp** (tương ứng kích thước bit của trường vô hướng $r = 512$ bit — số vòng lặp bằng độ dài bit của trường cơ sở p). Mỗi vòng lặp thực hiện một phép `double` và có thể một phép `add`, mỗi phép toán đó cần **1 phép nghịch đảo modulo p** trong hệ tọa độ Affine hiện tại. Phép nghịch

đảo được tính bằng định lý Fermat: $a^{-1} = a^{p-2} \pmod{p}$, tức là gọi đệ quy pow với khoảng **~1536 phép nhân Montgomery** 1024-bit bên trong. Đây là nguồn gốc của con số ~1.1–1.6 giây/phép nhân vô hướng.

Schnorr ký nhanh hơn ECDSA ~3% do Schnorr không cần tính nghịch đảo $k^{-1} \pmod{r}$ trong bước sinh chữ ký. Xác minh (cả Schnorr và ECDSA) tốn gần gấp đôi ký do cần **2 phép nhân vô hướng độc lập** ($[s]G$ và $[e]Q$ cho Schnorr; $[u_1]G$ và $[u_2]Q$ cho ECDSA). Tỷ lệ này hoàn toàn phù hợp với dự kiến lý thuyết.

So với secp256k1, độ chênh lệch (~1.5 s so với ~0.2 ms) xuất phát chủ yếu từ ba yếu tố tích lũy: (1) vòng lặp mul luôn chạy đủ 1024 bước do kiểu U1024, trong khi scalar của BLS12-381 chỉ có 255-bit có nghĩa — 769 vòng đầu luôn là bit 0 nhưng vẫn phải thực hiện phép double; (2) mỗi phép nhân Montgomery trên U1024 tốn ~16 lần công sức so với 256-bit (chi phí $O(n^2)$ với n số limb), bất kể p thực tế nhỏ hơn; và (3) secp256k1 có thư viện tối ưu nhiều năm với assembly AVX-512 được tinh chỉnh. Khoảng cách ~8000× là hệ quả trực tiếp của kiến trúc tổng quát ưu tiên tính đúng đắn, không phải dấu hiệu của thuật toán sai.

5.1.3. Tính khả thi trong ứng dụng thực tế

Với thời gian ~1.5–1.7 giây cho một phép ký, hệ thống hoàn toàn phù hợp với các **kịch bản ký ngoại tuyến** như ký tài liệu pháp lý, ký gói phần mềm khi phát hành, ký giao dịch tài chính giá trị lớn, hay ký chứng chỉ trong hạ tầng PKI — những trường hợp mà độ trễ vài giây là chấp nhận được và bù đắp hoàn toàn bởi biên bảo mật 256-bit so với 128-bit của các chuẩn thông thường.

Ngược lại, hệ thống **chưa phù hợp** cho các ứng dụng yêu cầu độ trễ dưới mili-giây như TLS handshake xử lý hàng nghìn kết nối/giây, giao dịch thanh toán tốc độ cao, hay ký batch thời gian thực. Đây là sự đánh đổi có chủ ý — tập trung vào tính đúng đắn và bảo mật của nền tảng toán học trước, tối ưu hóa sau.

Nhờ áp dụng **phép nhân Montgomery** thay cho phép chia lấy dư thông thường, tất cả phép nhân trong $\mathbb{F}_p$ đều thực thi bằng phép shift và trừ có điều kiện, không cần chia số 1024-bit cho p — một phép toán đắt hơn nhân ~5–10 lần. Đây là tối ưu quan trọng nhất đã được triển khai trong cài đặt hiện tại.

Các hướng cải tiến hiệu năng tiếp theo được phân tích tại Chương 6.

5.2. Đánh giá độ an toàn

5.2.1. An toàn trước tấn công cổ điển

Kiến trúc U1024 được thiết kế để hỗ trợ trường nguyên tố p lên đến 1024-bit và bậc nhóm r lên đến 512-bit, cung cấp mức bảo mật xấp xỉ 256-bit đối xứng khi triển khai đầy đủ. Bài toán ECDLP trên nhóm $E(\mathbb{F}_p)$ — tức là tìm k từ $Q = [k]G$ — hiện chưa có thuật toán tốt hơn thuật toán tổng quát. Thuật toán tốt nhất hiện tại là **Pollard's ρ** [7], có độ phức tạp $O(\sqrt{r}) \approx O(2^{256})$ phép toán nhóm. Với 2^{256} bước tính toán cần thực hiện, ngay cả khi huy động toàn bộ năng lực tính toán của nhân loại — mỗi nguyên tử Trái Đất thực hiện 10^9 phép tính mỗi giây từ khi Big Bang — vẫn không đủ để phá vỡ khóa trong tuổi thọ của vũ trụ. Đây là **biên an toàn tuyệt đối** trước mọi hệ thống máy tính cổ điển.

Mức bảo mật 256-bit vượt xa chuẩn tối thiểu NIST khuyến nghị (128-bit cho các ứng dụng đến năm 2030) và tương đương với AES-256 — chuẩn mã hóa đối xứng mạnh nhất hiện nay. Đây là mục tiêu thiết kế cốt lõi của thư viện 1024-bit này.

5.2.2. An toàn trước tấn công *Invalid Curve*

Invalid Curve Attack là một dạng tấn công nguy hiểm trong ECC: kẻ tấn công gửi một điểm P' không thuộc đường cong E nhưng vẫn hợp lệ về mặt định dạng, ép hệ thống tính toán trên một đường cong suy biến có bậc nhỏ, từ đó thu thập thông tin về khóa riêng qua nhiều truy vấn.

Hệ thống hiện tại **miễn nhiễm hoàn toàn** với tấn công này. Mọi điểm được tạo ra thông qua `AffinePoint::new(x, y)` đều tự động kiểm tra điều kiện:

$$y^2 \equiv x^3 + b \pmod{p}$$

bằng lời gọi `assert!(self.is_on_curve())` ngay trong hàm khởi tạo. Không tồn tại đường dẫn code nào cho phép một điểm không hợp lệ tồn tại trong hệ thống. Điều này được xác nhận bởi test `test_curve_point` trong bộ kiểm thử đơn vị.

5.2.3. An toàn trước tấn công *MOV*

Tấn công MOV (Menezes-Okamoto-Vanstone) [17] sử dụng phép ghép cặp (pairing) để ánh xạ bài toán ECDLP trong $E(\mathbb{F}_p)$ về bài toán DLP trong trường hữu hạn mở rộng $\mathbb{F}_{p^k}$, nơi bài toán DLP có thể được giải hiệu quả hơn bằng thuật toán chỉ số (index calculus). Tuy nhiên, hiệu quả của tấn công này phụ thuộc vào việc $\mathbb{F}_{p^k}$ có đủ nhỏ để bài toán DLP khả thi hay không. Kinh nghiệm cộng đồng yêu cầu $k|p| \geq 3072$ bit để kháng tấn công

này ở mức 128-bit [18]. Với các đường cong có kích thước trường cơ sở p lên tới 1024-bit hoặc có bậc nhúng k đủ lớn, hệ thống dễ dàng vượt xa tiêu chuẩn này. Kết quả này được test `test_mov_attack_resistance` xác nhận tường minh.

5.2.4. An toàn trước tấn công *Anomalous*

Tấn công Anomalous (SSSA) [19] khai thác trường hợp đặc biệt khi $\#E(\mathbb{F}_p) = p$, tức là số điểm trên đường cong bằng đúng modulus. Trong trường hợp này, tồn tại một đẳng cấu nhóm ánh xạ bài toán ECDLP về bài toán trên nhóm cộng $\mathbb{Z}/p\mathbb{Z}$, cho phép giải trong thời gian tuyến tính.

Curve1024 có $\#E(\mathbb{F}_p) = h \cdot r$ với r là bậc nhóm con nguyên tố 512-bit và h là cofactor, trong khi p là số nguyên tố 1024-bit. Hiển nhiên $\#E(\mathbb{F}_p) \neq p$ vì $|r| = 512 < 1024 = |p|$, nên điều kiện anomalous không thoả mãn. Test `test_anomalous_attack_resistance` xác minh điều này.

Lưu ý rằng các test `test_mov_attack_resistance` và `test_anomalous_attack_resistance` trong hệ thống hiện tại chỉ kiểm tra các điều kiện cần (như $r \neq p$), không phải điều kiện đủ đầy đủ cho việc kháng tấn công. Kiểm chứng toán học chặt chẽ hơn là hướng phát triển tiếp theo.

5.2.5. Hạn chế: *Timing Side-Channel Attack*

Đây là điểm cần trung thực học thuật: thuật toán **Double-and-Add** hiện tại, mặc dù có `conditional_select` ở từng bước đơn lẻ, vẫn có tổng số phép add thực hiện **phụ thuộc vào số lượng bit 1 trong scalar** k . Kẻ tấn công có thể đo thời gian thực thi để suy luận về trọng số Hamming của khóa riêng, từ đó thu hẹp không gian tìm kiếm.

Đây là hạn chế cần khắc phục trước khi đưa vào môi trường production. Giải pháp đã xác định là thay thế Double-and-Add bằng **Montgomery Ladder** — thuật toán thực hiện đúng số bước cố định bất kể giá trị scalar, đảm bảo tính *constant-time* thực sự.

Toàn bộ kết quả kiểm thử PASSED, bao phủ 17 test đơn vị và các test tích hợp, xác nhận tính đúng đắn toán học và an toàn của hệ thống trên mọi tiêu chí đã mô tả.

CHƯƠNG 6

KẾT LUẬN

6.1. Kết quả đạt được

Khóa luận đã hoàn thành mục tiêu đề ra: thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic hoàn chỉnh bằng ngôn ngữ Rust, với kiến trúc hỗ trợ trường nguyên tố tùy ý lên đến 1024-bit thông qua kiểu `U1024`, sử dụng tham số BLS12-381 làm baseline kiểm chứng, không phụ thuộc bất kỳ thư viện mật mã bên ngoài nào. Các đóng góp cụ thể bao gồm:

- **Lớp số học bignum an toàn bộ nhớ:** Tự định nghĩa kiểu dữ liệu `U1024` với phép cộng, trừ, nhân, lũy thừa và nghịch đảo. Phép nhân Montgomery [15] được cài đặt trực tiếp, loại bỏ hoàn toàn phép chia số lớn trong vòng lặp tính toán, đồng thời bảo đảm an toàn bộ nhớ tuyệt đối nhờ kiến trúc Rust.
- **Đường cong elliptic bằng phương pháp CM:** Cài đặt phương pháp Nhân phức (CM) hỗ trợ sinh tham số đường cong trên trường nguyên tố tùy ý (xem `examples/complex_multiplication.rs`), kiểm chứng chéo tính đúng đắn thư viện bằng đường cong chuẩn BLS12-381 [16].
- **Hai sơ đồ chữ ký số chuẩn mực:** Schnorr [11] và ECDSA [10] được cài đặt đầy đủ và xác minh đúng đắn.
- **Bộ kiểm thử toàn diện:** 36/36 test PASSED, bao phủ từ số học cấp thấp (`U1024`, `FieldElement`) đến an toàn mật mã cấp cao (mô phỏng tấn công MOV, Anomalous, Invalid Curve).

6.2. Đánh giá tổng quan

6.2.1. So sánh với mục tiêu ban đầu

Nhìn lại bốn mục tiêu đặt ra tại Chương 1, khóa luận đạt được kết quả như sau:

Bảng 6.1. Đối chiếu kết quả với mục tiêu đề ra

Mục tiêu	Kết quả
Xây dựng đường cong elliptic	✓ Phương pháp CM, kiểm chứng chéo bằng BLS12-381
Bảo mật trước máy tính cổ điển	✓ Bảo mật 256-bit, vượt xa chuẩn NIST 128-bit đến năm 2030
An toàn bộ nhớ ngôn ngữ	✓ Rust loại bỏ toàn bộ lỗi bộ nhớ tại thời điểm biên dịch
Chữ ký số vận hành được	✓ Schnorr và ECDSA ký/xác minh chính xác, 36/36 test PASSED

6.2.2. Đánh giá điểm mạnh và hạn chế

Điểm mạnh:

Hệ thống đạt được tính *đúng đắn toán học* cao — mọi công thức từ phép cộng điểm affine, phép nhân Montgomery đến quy trình ký/xác minh Schnorr và ECDSA đều được kiểm thử độc lập. Tính *minh bạch thuật toán* là ưu điểm nổi bật so với các thư viện “hộp đen” công nghiệp: toàn bộ tham số đường cong được kiểm chứng bằng phương pháp toán học độc lập (CM + cross-validation với BLS12-381).

Hạn chế:

Hiệu năng hiện tại (~1.5 s/phép ký) chưa đủ cho các ứng dụng yêu cầu độ trễ thấp như TLS hay thanh toán thời gian thực. Đây là sự đánh đổi có chủ ý khi ưu tiên tính đúng đắn và kiến trúc phân tầng rõ ràng trước, tối ưu hóa sau. Ngoài ra, thuật toán Double-and-Add hiện tại tiềm ẩn nguy cơ timing side-channel do số phép `add` phụ thuộc vào trọng số Hamming của scalar — một điểm yếu cần giải quyết trước khi đưa vào môi trường production.

6.3. Hướng phát triển

6.3.1. Tối ưu hóa hiệu năng (ngắn hạn)

1. **Tọa độ Jacobian/Projective:** Loại bỏ phép nghịch đảo tốn kém ($a^{p-2} \bmod p$) trong mỗi bước `add/double` bằng cách biểu diễn điểm ở dạng $(X : Y : Z)$ thay vì (x, y) Affine. Toàn bộ vòng lặp nhân vô hướng 1024 bước chỉ cần 1 phép nghịch đảo duy nhất ở cuối — ước tính giảm thời gian ký từ ~1.5 s xuống dưới **200 ms**.
2. **Sliding Window / NAF:** Giảm ~25–30% số phép `add` so với Double-and-Add cơ

bản bằng cách nhóm nhiều bit scalar lại. Kết hợp với Jacobian, thời gian ký dự kiến đạt dưới **150 ms**.

3. **AVX-512 / SIMD assembly:** Vector hóa các phép nhân 64×64 -bit limb trong Montgomery multiplication trên kiến trúc x86-64. Thư viện `blst` của Ethereum đạt tốc độ BLS12-381 signing dưới 1 ms nhờ kỹ thuật này — con số tham chiếu thực tế cho hướng tối ưu tiếp theo.

6.3.2. Bảo mật cấp cao (trung hạn)

4. **Montgomery Ladder:** Thay thế Double-and-Add bằng Montgomery Ladder để đạt *constant-time* thực sự — thuật toán luôn thực hiện đúng 1024 phép `double` và 1024 phép `add` bất kể giá trị scalar, loại bỏ hoàn toàn timing side-channel.
5. **Multi-Scalar Multiplication (MSM — Pippenger):** Tối ưu bước xác minh $[s]G + [e]Q$ bằng thuật toán Pippenger [16], giảm ~50% công việc so với hai phép nhân vô hướng tuần tự.

6.3.3. Mở rộng ứng dụng (dài hạn)

6. **Giao thức ngưỡng (Threshold Signatures):** Dựa trên tính tổng hợp tự nhiên của Schnorr, cài đặt các giao thức MuSig2 hay FROST cho phép t -trong- n người ký tham gia, phù hợp với ứng dụng đa chữ ký (multi-sig) trong blockchain và hệ thống tài chính.
7. **Kiểm định hình thức (Formal Verification):** Sử dụng framework như `creusot` hoặc `kani` để chứng minh tính đúng đắn của các tính chất quan trọng (ví dụ: `add` thỏa mãn luật giao hoán và kết hợp) ở mức ngôn ngữ hình thức, nâng mức độ đảm bảo an toàn lên chuẩn mực cao nhất.

Các hướng **1** và **4** là ưu tiên cao nhất vì chúng đồng thời cải thiện cả hiệu năng lẫn bảo mật mà không đòi hỏi thay đổi kiến trúc cấp cao, và có thể được triển khai độc lập trong phạm vi codebase hiện tại.

TÀI LIỆU THAM KHẢO

- [1] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi: 10.1109/TIT.1976.1055638.
- [2] Michael Rosing. *Elliptic Curve Cryptography for Developers*. Manning Publications, 2024. ISBN 9781633437845.
- [3] Elaine Barker. Recommendation for key management, part 1: General. Technical Report NIST Special Publication 800-57 Part 1 Revision 5, National Institute of Standards and Technology, 2020.
- [4] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.
- [5] Victor S. Miller. Use of elliptic curves in cryptography. *Advances in Cryptology – CRYPTO ’85*, pages 417–426, 1986.
- [6] Dan Shumow and Niels Ferguson. On the possibility of a back door in the nist sp800-90 dual ec prng. In *CRYPTO 2007 Rump Session*, 2007.
- [7] John M. Pollard. Monte carlo methods for index computation $(\text{mod } p)$. *Mathematics of Computation*, 32(143):918–924, 1978.
- [8] CVE. Cve-2014-0160: The heartbleed bug. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160>, 2014.
- [9] Matt Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, 2019. https://github.com/Microsoft/MSRC-Security-Research/blob/master/presentations/2019_02_BlueHatIL/Trends,%20challenge,%20and%20shifts%20in%20vulnerability%20mitigation.pdf.
- [10] NIST. Digital signature standard (dss). Technical Report FIPS PUB 186-4, National Institute of Standards and Technology, 2013.

- [11] C. P. Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology – CRYPTO ’89*, pages 239–252. Springer, 1990. doi: 10.1007/0-387-34805-0_22.
- [12] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*. Springer, 2nd edition, 2009. doi: 10.1007/978-0-387-09494-6.
- [13] Lawrence C. Washington. *Elliptic Curves: Number Theory and Cryptography*. Chapman and Hall/CRC, 2nd edition, 2008. ISBN 9781420071467.
- [14] Tran Duy. curve1024: Thư viện mật mã đường cong elliptic 1024-bit viết bằng rust. <https://github.com/Tranduy1d01/curve1024>, 2026. Mã nguồn mở, MIT License.
- [15] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985. doi: 10.1090/S0025-5718-1985-0777282-X.
- [16] Sean Rowe. Bls12-381: New zk-snark elliptic curve construction. <https://electriccoin.co/blog/new-snark-curve/>, 2017.
- [17] Alfred J. Menezes, Tatsuaki Okamoto, and Scott A. Vanstone. Reducing elliptic curve logarithms to logarithms in a finite field. In *IEEE Transactions on Information Theory*, volume 39, pages 1639–1646, 1993. doi: 10.1109/18.259647.
- [18] Razvan Barbulescu and Sylvain Duquesne. Updating key size estimations for pairings. *Journal of Cryptology*, 32:1298–1336, 2019. doi: 10.1007/s00145-018-9280-5.
- [19] Nigel P. Smart. The discrete logarithm problem on elliptic curves of trace one. *Journal of Cryptology*, 12(3):193–196, 1999. doi: 10.1007/s001459900052.